AI Tools for Biomedical Researchers: A Hands-On Workshop

Sep 24, 2026 · @Marcus

At a glance

A three-session, hands-on workshop that makes graduate students and early-stage researchers comfortable using AI tools in biomedical research. It is designed for wet-lab researchers and also useful for early-stage dry-lab students; minimal Python is assumed. About two-thirds of each session is hands-on, and one biological thread, p53 and MDM2, runs through all three days.

By the end, students can explain how AI fails, verify its output, connect their own data to an LLM, direct a coding agent through a full analysis, and apply rules for appropriate AI use in research.

Format	3 weekly sessions × 3.5 hours (same weekday, three consecutive weeks); one Jetstream2 VM per student (JupyterLab, Open WebUI, hux), kept running from Day 1 until one week after Day 3
Cohort and staff	20–25 students per session; instructor plus 1–2 TAs
Prerequisites	Laptop and browser; able to run a notebook cell and edit one line of Python
Resources	Explore ACCESS allocation (~75,000 SUs); up to 26 VMs; public data only
Dates	To be scheduled: one weekday, three consecutive weeks

Schedule at a glance

Day	Theme	Session flow (clock time from start)	Students build
1	How AI Works (and Where It Breaks)	0:00 Welcome · 0:05 Warm-up · 0:20 Lecture · 0:55 Lab A: TP53 classifier · 1:50 Break · 2:00 Lab B: LLM sampling and the agent loop · 2:50 Bring your own · 3:00 Debrief · 3:20 Bridge	A classifier; a traced agent run
2	Making Your Data Available to an LLM	0:00 Opening · 0:15 Lecture · 0:45 Lab A: retrieval index · 1:40 Break · 1:50 Lab B: MCP server connected to Open WebUI · 2:50 Bring your own · 3:00 Debrief · 3:20 Bridge	A hybrid retrieval index; an MCP server in their own chat interface
3	AI-Assisted Coding for Bench Scientists	0:00 hux orientation · 0:20 Lecture · 0:50 Lab 1: plan, critique, QC · 1:40 Break · 1:50 Lab 2: DE, figures, enrichment, report · 2:45 Bring your own · 2:55 Debrief · 3:05 Appropriate AI use · 3:25 Close	A full RNA-seq analysis directed through hux

Detailed schedules, the appropriate-use framework, resources, and the preparation checklist follow.

Contents

1. Course summary
2. Prerequisites, environment, and policies
3. Day 1: How AI Works (and Where It Breaks)
4. Day 2: Making Your Data Available to an LLM
5. Day 3: AI-Assisted Coding for Bench Scientists
6. Appropriate use of AI in research

7. Resources required
8. Instructor preparation and open decisions

Course summary

A three-session, hands-on workshop that makes graduate wet-lab students comfortable using AI tools in their own research. About two-thirds of every session is hands-on: students learn comfort by doing.

Audience	Graduate students and early-stage researchers. Designed for wet-lab researchers; also useful for early-stage dry-lab students. Minimal Python, no prior AI/ML experience
Cohort	Planned for 20–25 students per session
Format	3 sessions × 3.5 hours, one per week on the same weekday over three consecutive weeks; Python in Jupyter notebooks, a personal Open WebUI chat interface, and the hux coding agent
Environment	One Jetstream2 VM per student, kept running for one week after the workshop
Instructor	Marcus R. Breese, PhD, Department of Medical and Molecular Genetics, IU School of Medicine

Learning outcomes

By the end of the workshop, students can:

1. Explain how classifiers and LLMs learn from examples, and predict where each one fails.
2. Recognize and verify common LLM failures: fabricated citations, confident errors, and tokenization blind spots.
3. Make their own data available to an LLM by building a retrieval index and an MCP server, and choose between prompt stuffing, retrieval, and tools.
4. Direct a coding agent through a complete RNA-seq analysis from a written plan with pre-registered checks, and catch the errors the agent misses.

5. Apply a framework for appropriate AI use in research, and write an AI-use disclosure.

Common thread: p53 and MDM2

All three days use the same biology. Day 1 predicts TP53 status from gene expression. Day 2 serves cell-line and literature data about p53 to an LLM. Day 3 analyzes the transcriptional response to nutlin, an MDM2 inhibitor, in TP53 wild-type vs. knockout cells.

Three-day arc

Day	Title	Students build	Students leave with
1	How AI Works (and Where It Breaks)	A TP53-status classifier; a traced agent run	An intuition for how AI fails, and a habit of verifying
2	Making Your Data Available to an LLM	A hybrid retrieval index and an MCP server, connected to their own chat interface	Their own data and tools inside a working chat app
3	AI-Assisted Coding for Bench Scientists	A full bulk RNA-seq analysis, directed through hux	A plan template, review checklist, reproducible notebook, and AI-use disclosure

Design principles

- **Do first, explain second.** Each day opens with a hands-on task; the lecture explains what students just saw.
- **Use and verify, build the data side.** Students use provided AI interfaces and build only the pieces that connect their data: a classifier, a retrieval index, an MCP server.
- **Bring your own.** Each day ends with students applying that day's tools to their own research.
- **Public data only.** All workshop data is public; sensitive data stays in IU-approved tools.

Prerequisites, environment, and policies

Students need a laptop with a modern browser and enough Python to run a notebook cell and edit one line. Nothing is installed locally; everything runs on a personal VM.

What students bring

Day	Bring
1	A question from your own research to ask an AI
2	A CSV from your lab (sample inventory, plate layout, reagent list); public or non-sensitive only. A fallback CSV is provided
3	An upcoming experiment you want to plan an analysis for

Before the workshop

- Complete a short survey: Python comfort, and whether you work with bulk or single-cell RNA-seq.
- Nothing else. Credentials are handed out on a printed card at the start of Day 1.

Computing environment

Each student gets a VM at `vmN.class.compgenlab.org` and logs in as `studentN@class.compgenlab.org` with a generated password.

Component	Used on	Purpose
JupyterLab	Days 1–3	Notebooks and terminal
Open WebUI (student is admin)	Days 1–2	Chat interface; students connect their own MCP servers
Student MCP server (run from the command line)	Days 2–3	The student's own tools: cell-line lookups and literature search
hux coding agent	Day 3	Directed analysis in a container
Workshop model (Jetstream-hosted)	Days 1–3	The LLM behind every interface
Workshop webapp (instructor VM)	Days 1–3	Live polls, charts, and anonymous votes

Policies

- **Public data only.** Jetstream2 does not permit data protected by law (HIPAA, FERPA and similar) without specific administrative authorization. Use IU's REALLMS for anything sensitive.
- **VM lifetime.** VMs stay up for one week after Day 3, then are deleted. Export anything you want to keep: notebooks, server code, and chat exports.
- **Anonymous polling.** All in-class votes and submissions are anonymous.
- **Appropriate AI use.** The framework taught on Day 3 applies to all workshop work.

Day 1: How AI Works (and Where It Breaks)

Students build a classifier, watch an LLM sample and an agent loop run, and measure where both fail. About 62% hands-on.

Clock	Block	Min	Mode
0:00	Welcome: workshop goal and the p53/MDM2 thread	5	Talk

Clock	Block	Min	Mode
0:05	Setup and warm-up	15	Hands-on
0:20	Lecture	35	Talk
0:55	Lab A: build a classifier	55	Solo
1:50	Break	10	
2:00	Lab B: inside an LLM and the agent loop	50	Follow-along, then solo
2:50	Bring your own	10	Solo
3:00	Debrief	20	Discussion
3:20	Bridge to Day 2	10	

Setup and warm-up (15 min)

1. Log in to JupyterLab and Open WebUI with the credentials card. TAs fix failures on the spot.
2. In Open WebUI, ask a question from your own research and read the answer.
3. Mark one factual claim in the answer; you will verify it at the end of the day.
4. Optional (2 min): vote on the 10 appropriate-use scenarios, for comparison on Day 3.

Lecture (35 min)

Each segment starts from something students saw in the warm-up.

Segment	Min	Content
Two kinds of AI	4	Learning from examples; discriminative vs. generative models
Classifiers	10	Features, labels, train/test split, overfitting, confusion matrix, ROC
Neural networks	3	Intuition only: stacked learned features

Segment	Min	Content
How LLMs work	10	Tokens; the next-token distribution; predict → sample → append; temperature; context windows. Same training recipe as a classifier, different target; the loop makes it generative
Where it breaks	4	Forced choice (always an answer, never "I don't know"); hallucination; tokenization blind spots
Agents	4	Agent = LLM + tools + loop; the harness is the loop

Lab A: Predict TP53 status from expression (55 min, solo)

Data: a DepMap subset of cancer cell lines with expression, TP53 status, and lineage, in two versions: a panel of ~15 canonical p53 targets (CDKN1A, MDM2, BAX, BBC3, GDF15, SESN1, RRM2B, TP53I3, ZMAT3, TNFRSF10B, DDB2, PHLDA3 and others) and all genes.

Step	Min	Task
A1	10	Explore: class balance; CDKN1A expression by TP53 status
A2	15	Logistic regression on the panel: confusion matrix, ROC, coefficients. Checkpoint: fitted model
A3	25	Break it: all genes vs. panel (overfitting); feature selection before the split (leakage); train on 3 lineages and test on the rest (confounding)
A4	5	Two sentences: what did the model learn, and how do you know?

Stretch: random forest vs. logistic regression. Held for the debrief: misclassified MDM2-amplified wild-type lines.

Lab B: Inside an LLM and the agent loop (50 min)

Step	Mode	Min	Content
B1	Follow-along	10	In the notebook, print next-token probabilities for "Name one p53 target gene. Answer with the gene symbol only." Each student samples ~5 answers at T = 0, 0.7 and 1.2 and submits them; the webapp charts the room's answers live beside the model's probabilities
B2	Instructor-led	12	Walk through one agent trace in Open WebUI: question → decision → tool call → result → answer. Students ask their own question with code interpreter on and label each step of their trace
B3	Solo	13	Ask "What is the TP53 status of SJSA-1?" with no tools, then with the reference cell-line server enabled. Compare answers and traces
B4	Solo	15	Probes: count the Gs in a TP53 exon with and without code interpreter; ask for 3 PMIDs on p53–MDM2 with no tools, look each up in PubMed, classify it (fabricated / real but unrelated / supporting), and submit to the webapp

Bring your own (10 min)

Verify the claim you marked in the warm-up. Then re-ask the question requiring citations or with code interpreter on: did the answer improve, and can you verify it?

Debrief (20 min)

Classifier	LLM
Always outputs one of its labels, even on unfamiliar input	Always produces a fluent answer, even for facts that don't exist
Leakage inflates accuracy	Fluency inflates confidence
Learns lineage instead of biology	Can't count characters without a tool

Each student names one failure they found. Close on the MDM2-amplified lines: the model read p53 pathway activity correctly; the label was "wrong." This sets up nutlin on Day 3.

Bridge to Day 2 (10 min)

1. Show the room's PMID results.
2. Ask: "Same question next week, but the agent can search the literature. What happens to this chart?" Students submit a predicted fabrication rate.
3. Remind students to bring a CSV from their lab. One-minute feedback card.

Day 2: Making Your Data Available to an LLM

Students build the data side, a hybrid retrieval index and an MCP server, and connect it to their own Open WebUI. They then measure prompt stuffing, retrieval, and tools on the same questions. About 67% hands-on.

Clock	Block	Min	Mode
0:00	Opening: the reveal and a question the agent can't answer	15	Hands-on
0:15	Lecture	30	Talk
0:45	Lab A: build a retrieval index	55	Mixed
1:40	Break	10	
1:50	Lab B: build and connect an MCP server	60	Mixed
2:50	Bring your own	10	Solo
3:00	Debrief	20	Discussion
3:20	Bridge to Day 3	10	

Opening (15 min)

1. In Open WebUI, ask: "What is the TP53 status of SJSA-1, and how many osteosarcoma lines are TP53 wild-type?" The model guesses or refuses; that is the problem the day solves.

2. Show the Day 1 PMID chart beside the class's predicted fabrication rates for today. We return to it once retrieval is connected.

Lecture (30 min)

Segment	Min	Content
The problem	3	The model knows only its training data and its context window
Stuffing	4	Paste data into the prompt: simple, but limited by context length, and recall degrades as it grows
Retrieval (RAG)	9	Embeddings are learned features, like Day 1's classifier inputs. Chunk → embed → retrieve → answer. Embeddings handle paraphrase; keyword search handles exact identifiers; hybrid combines both. Good for citations, bad for counting
Tools and MCP	8	A tool is a function; MCP is a standard way to serve tools to any client (host, client, server, transports)
Where your data goes	3	Your VM and the workshop model vs. commercial connectors; the server is the policy boundary
Injection	3	Retrieved text and tool output become model input, so a document can carry instructions

Lab A: Build a retrieval index (55 min)

Step	Mode	Min	Content
A1	Solo	10	Stuffing: paste 100, 1,000, and all rows of the cell-line table into Open WebUI; ask 4 lookups at each size and record accuracy
A2	Follow-along	20	In the notebook, build two indexes over ~300 p53/MDM2 abstracts: embeddings (numpy + a small embedding model) and keyword search (BM25). Search both; save to disk

Step	Mode	Min	Content
A3	Solo	15	Find a failure: query an exact identifier (a gene symbol such as PHLDA3, or a cell line such as SJSA-1). Watch embedding search miss and BM25 find it, then combine the scores into a hybrid search
A4	Solo	10	Buffer, or stretch: change the chunk size and compare

Checkpoint: prebuilt index files. Stretch: upload the same abstracts to Open WebUI's built-in Knowledge feature and compare its retrieval with yours.

Lab B: Build and connect an MCP server (60 min)

Step	Mode	Min	Content
B1	Follow-along	15	Write a FastMCP server with `lookup_cell_line` (Day 1's tool) and `search_literature` (your hybrid search); start it in a JupyterLab terminal with `python server.py`
B2	Solo	10	Add `count_cell_lines(filter)`; stop the server (Ctrl-C) and start it again
B3	Follow-along	10	In Open WebUI: Settings → Admin → Integrations → External Tool Servers → add MCP (Streamable HTTP) with your server URL and auth None. Add paperKB the same way. Enable both in a chat
B4	Solo	25	Score 6 questions by hand under stuffing, retrieval only, and all tools, and submit to the webapp. Rerun the Day 1 PMID question for the before/after chart. Injection probe: query the planted abstract and see whether the agent follows its instruction

Question set: 2 lookups, 2 aggregates, 1 literature question, and 1 that only paperKB can answer. Checkpoint: a known-good server file.

Bring your own (10 min)

Add one tool that reads a CSV from your own lab, stop and restart the server in its terminal, and ask about it in Open WebUI. A plate-layout CSV is provided as a fallback.

Debrief (20 min)

- Accuracy matrix (method × question type): stuffing degrades with size; retrieval wins on literature but not aggregates; tools win on lookups and aggregates; paperKB wins on scale.
- PMID before/after: fabrication drops with retrieval, but "real but unrelated" citations remain.
- Injection results: who was affected, and through which method.
- Where to go next: biomedical embedding models (e.g., MedCPT), and paperKB as the production version of what students built.
- Optional instructor demo (5 min): the same server connected to a second client, showing MCP's portability.

Bridge to Day 3 (10 min)

1. Next week: a full RNA-seq analysis of nutlin vs. DMSO in TP53 wild-type vs. knockout cells.
2. Students pre-register 3 genes they expect to rise in wild-type but not knockout cells, and submit them to the webapp.
3. Leave the server running in its JupyterLab terminal, or start it again at the beginning of Day 3. Feedback card.

Day 3: AI-Assisted Coding for Bench Scientists

Students direct a coding agent through a full bulk RNA-seq analysis: the student owns the plan and verification, the agent writes the code, and planted errors test whether the plan catches what an unplanned request misses. About 70% hands-on.

Dataset: GSE86221, HCT116 TP53 wild-type vs. knockout, each treated with DMSO or nutlin (a 2×2 design). The instructor provides gene-level counts; no alignment in class.

Clock	Block	Min	Mode
0:00	Opening: hux orientation and an unplanned request	20	Hands-on
0:20	Lecture	30	Talk
0:50	Lab part 1: plan, critique, QC	50	Solo
1:40	Break	10	
1:50	Lab part 2: DE, figures, enrichment, literature, report	55	Solo
2:45	Bring your own	10	Solo
2:55	Debrief	10	Discussion
3:05	Appropriate use of AI in research	20	Interactive
3:25	Workshop close	5	

Opening (20 min)

1. Terminal orientation (10 min): open a terminal in JupyterLab and start your Day 2 server if it isn't running; open a second terminal and start hux; where files live; how to stop the agent.
2. The unplanned request (10 min): type "Analyze the RNA-seq data in `data/`" and watch. Expect plausible output that computes straight through the planted errors. Save it for the debrief.

Lecture (30 min)

Other AI Use Cases in Biology (20 min):

Segment	Min	Content	p53 tie-in
Image classification	5	Dog vs. cat → tumor vs. normal → microscopy; Cellpose as a tool to use next week; shortcut learning	p53 IHC scoring

Segment	Min	Content	p53 tie-in
Protein models	5	ESM-2 (embeddings for classifiers, zero-shot variant scoring); AlphaFold 2/3	TP53 hotspots; the MDM2–nutlin pocket
Genome models	5	DNA language models (Evo 2) vs. sequence-to-function models (AlphaGenome)	p53 response elements; MDM2 SNP309
Evaluating any AI tool	5	Training data? Beats a simple baseline on data like mine? How does it fail? Can I use it? How would I verify it? (Example: single-cell foundation models often fail the baseline test)	

Plan → implement → verify (10 min): the opening agent was fast, confident, and wrong in ways it never flagged. The scientist owns the question, the checks, and the conclusions; the agent owns the typing. A plan includes pre-registered known answers.

Where agents are heading (mention only, ~2 min of the 10; nothing hands-on):

- **Skills:** packaged instructions, scripts, and reference files an agent loads only when a task calls for them, such as "run differential expression the way our lab does it." They let a lab encode its conventions once instead of re-prompting every time.
- **Multi-agent workflows:** an orchestrating agent delegates to specialized sub-agents, for example one that plans, one that writes code, and one that reviews it, sometimes running in parallel.
- **The same rule applies, harder:** more autonomy means more places for errors to compound between steps, so a written plan with checks matters more, not less.

Lab part 1 (50 min)

Step	Min	Content
L1	20	Write the plan, no AI. Template: question, comparisons, known-answer checks (CDKN1A and MDM2 rise with nutlin in wild-type, not knockout; replicates cluster by condition; Hallmark p53 pathway enriched in wild-type only; your Day 2 predictions), figures
L2	10	hux critiques the plan; revise. Expect it to raise the genotype × treatment interaction
L3	20	QC with hux: library sizes, sample correlation, PCA. Planted errors: a DMSO/nutlin label swap (visible in PCA and in MDM2/CDKN1A), and one column of normalized values instead of raw counts (non-integers). Checkpoint: corrected counts and sample sheet

Lab part 2 (55 min)

Step	Min	Content
L4	15	DE with PyDESeq2: nutlin vs. DMSO within each genotype, plus the interaction. Check the known answers. Planted error: Ensembl ID version suffixes differ between tables, so genes silently drop from the join; check row counts
L5	10	Figures: volcano plots, top-gene heatmap, known-target plots
L6	10	Enrichment: gseapy prerank against offline Hallmark gene sets
L7	10	Literature check in Open WebUI (your Day 2 server + paperKB): are the top wild-type-specific genes outside the Hallmark set known p53 targets? Reveal Day 2 predictions vs. actual results

Step	Min	Content
L8	10	Report: a notebook that reruns end to end. The student writes the methods and conclusions; hux checks the methods against the code (parameters, versions, filters) and flags discrepancies

Rule for the whole lab: read each step's output and run its check before moving on. Checkpoints after every step.

Extension track (fast finishers and the week after): the MIX-seq idasanutlin single-cell data, 24 cell lines with known TP53 status. The plan must pre-register pseudobulk DE by cell line; if the agent tests on individual cells, stop.

Bring your own (10 min)

Draft an analysis plan for your own next experiment with the same template; hux critiques it.

Debrief (10 min)

For each planted error: caught or not, and by what (a plan check, the student, or the agent)? Compare with the unplanned request from the opening. Hand out the code-review checklist.

Appropriate use of AI in research (20 min)

Part	Min	Content
Gray areas	2	Students anonymously submit a situation from their own lab where they weren't sure AI use was OK
Scenario vote	10	Vote on 10 scenarios (OK / OK with conditions / Not OK); show the split, discuss, give the framework's answer; re-vote on the two hardest; discuss one or two submitted gray areas
Framework	3	The one-page handout (see Appropriate use section)

Part	Min	Content
Disclosure	5	Each student drafts an AI-use statement for their Day 3 analysis

Workshop close (5 min)

VMs stay up one week (state the shutdown date). Public data only; REALLMS for anything sensitive. Students keep notebooks, server code, chat exports, the plan template, checklist, and disclosure. Final survey.

Appropriate use of AI in research

The rule behind every row: AI can do the work, but you stay accountable for every result, citation, and word, and you disclose how you used it. The framework is introduced across all three days and consolidated on Day 3.

Day	Moment	Rule it illustrates
1	PMID probe (fabricated citations)	Never cite what you haven't read and verified
2	"Where your data goes"	Sensitive or unpublished data only goes into approved tools
3	Planted errors; plan-then-verify	You are accountable for every analysis step, whoever wrote the code

Framework (handout)

Use	Status	Condition
Coding	OK	You review it, test it, and can explain it
Orchestrating analysis	OK	You own the plan and the checks; keep agent logs as provenance; no silent exclusions
Literature search and summary	OK	Read and verify every citation before using it

Use	Status	Condition
Writing	Editing only	The ideas and drafts are yours; AI polishes; disclose per journal policy
Data	Analysis only	Never generate, impute, or alter data presented as measured; no AI-generated images of results
Sensitive data	Approved tools only	PHI and unpublished data go to IU-approved tools such as REALLMS, not Jetstream2 or commercial chatbots
Peer review	Not OK	Manuscripts and grants under review are confidential (NIH NOT-OD-23-149)
Grant applications	Editing only	NIH will not consider applications substantially developed by AI to be the applicant's original ideas (NIH NOT-OD-25-132)

Align the handout with IU's generative AI guidance and cite it directly. Re-check the NIH notices the week before delivery.

Voting scenarios

#	Scenario	Framework answer
1	The agent writes your DESeq2 code; you review it and run the checks	OK
2	AI fills in a missing replicate so the statistics run	Not OK: fabrication
3	AI rewrites your discussion section for clarity	OK, with disclosure
4	AI drafts your Specific Aims from a one-line idea	Not OK for NIH
5	You paste a manuscript you're reviewing into ChatGPT for a summary	Not OK: confidentiality

#	Scenario	Framework answer
6	AI generates a "representative" western blot	Not OK: fabrication
7	You cite three papers an AI listed without reading them	Not OK
8	AI denoises microscopy images for a figure	Conditions: disclose, apply uniformly, keep raw data; journal policies vary
9	The agent silently drops two outlier samples	Not OK unless pre-specified and reported
10	You upload patient data to a commercial chatbot	Not OK

Voting mechanics: anonymous; reveal the room's split before the answer; re-vote on the two scenarios with the closest split after discussion. If students also voted during Day 1 setup, show the Day 1 and Day 3 distributions side by side.

Resources required

The workshop needs one Explore ACCESS allocation (~75,000 SUs requested), up to 26 Jetstream2 VMs, one to two TAs, and public datasets staged in advance.

Compute: Jetstream2 via an Explore ACCESS allocation

On Jetstream2, 1 ACCESS credit = 1 SU = 1 vCPU-hour. Explore allocations cover up to 400,000 credits and are intended for small-scale classroom use (Jetstream2 allocations).

Item	Flavor	Count	Duration	SUs
Student VMs	m3.quad (4 vCPU, 15 GB)	25	22 days (Day 1 through Day 3, then 7 after)	52,800
Instructor VM (webapp, reverse proxy, reference server)	m3.medium (8 vCPU)	1	33 days	6,336
Rehearsal VMs	m3.quad	3	3 days	864

Item	Flavor	Count	Duration	SUs
Total with 25% headroom				~75,000

Request steps: submit the request with a CV and this syllabus as PDF; allow 1–2 business days for approval; exchange credits for Jetstream2 compute; add TAs as allocation managers. Students do not need ACCESS accounts, because the instructor provisions their VMs. Upgrade the instructor VM to m3.large (+~6,300 SUs) if paperKB also runs there.

Networking and provisioning

- One public IP on the instructor VM; wildcard DNS `*.class.compgenlab.org` pointing to it; a wildcard Let's Encrypt certificate via DNS challenge.
- The reverse proxy routes `vmN.class.compgenlab.org` to each student VM's private IP.
- Terraform (`count = final enrollment, up to 25`) plus cloud-init builds every VM; `terraform destroy` on the shutdown date.
- Open WebUI signup disabled; each instance's admin account is pre-created as `studentN@class.compgenlab.org` with a generated password.

VM image contents

Component	Configuration
JupyterLab	Workshop notebooks, `labagent` helpers, checkpoint files
Open WebUI (latest; v0.6.31+ required for native MCP)	Workshop model preconfigured; `WEBUI_SECRET_KEY` set; code interpreter on; Docker run with `--add-host=host.docker.internal:host-gateway`; reference cell-line server pre-registered
Student MCP server	No service: the student starts the server from a JupyterLab terminal (`python server.py`, port 8000). JupyterLab terminals keep running when the browser tab closes, but the server stops if the terminal is closed or the VM reboots

Component	Configuration
hux	Configured to the workshop model; Docker as container runtime
Python container	PyDESeq2, gseapy, scikit-learn, rank_bm25, numpy/pandas, matplotlib/seaborn, FastMCP, a small embedding model (pre-cached weights)

Data

Day	Dataset	Source	Preparation
1	Cancer cell-line expression, TP53 status, lineage	DepMap (CC BY 4.0)	Panel and all-genes versions; pre-verify leakage and lineage effects are visible
1	TP53 exon sequence	NCBI	Probe input
2	~300 p53/MDM2 abstracts	PubMed (open access)	Plus one planted injection abstract; prebuilt index checkpoint
2	Cell-line annotation and curated p53 target table	DepMap and literature	Served by the student and reference servers
2–3	Frozen literature collection	paperKB (open-access subset)	MCP over streamable HTTP; workshop keys; load test at 25 clients
3	Bulk RNA-seq, HCT116 TP53 WT/KO × DMSO/nutlin	GSE86221	Instructor aligns and counts; planted-error version plus clean answer key
3	MSigDB Hallmark gene sets	MSigDB	Staged offline
3 (extension)	MIX-seq idasanutlin single-cell	pertpy / scPerturb	Pre-staged AnnData with its own plan template

Staffing

- Instructor: lectures, follow-along coding, debriefs.
- TAs: one to two for 20–25 students, plus an on-call contact for the week after.

Materials

- Credentials cards (hostname, login, password, shutdown date) and a master sheet for TAs.
- Handouts: plan template, code-review checklist, appropriate-use framework, disclosure example.
- Workshop webapp views: sampling chart, PMID before/after, accuracy matrix, Day 3 predictions, scenario vote with re-vote, gray-area submissions.
- Pre-baked lecture visuals: Cellpose segmentation, AlphaFold 3 MDM2–nutlin structure, AlphaGenome track.
- Fallback plate-layout CSV for Day 2.

Instructor preparation and open decisions

The longest lead items are the VM build and a full rehearsal on the workshop model; the allocation itself takes days, not weeks.

Checklist

6+ weeks before

- [ ] Request the Explore ACCESS allocation; exchange credits for Jetstream2 compute; add TAs
- [ ] Send the pre-workshop survey (Python comfort; bulk vs. single-cell)
- [ ] Align and count GSE86221; confirm replicates per condition; build the planted-error and clean versions
- [ ] Freeze the paperKB open-access collection and finalize its MCP tool surface

4 weeks before

- [ ] Build the VM image and Terraform/cloud-init; stand up the instructor VM, DNS, and wildcard certificate
- [ ] Build `labagent` helpers (model calls, `submit()`, chunk/embed/BM25/search), the server scaffold, and solution files

- [ ] Build the workshop webapp and all its views; load-test ~375 submissions in a burst
- [ ] Prepare the DepMap subset, abstract corpus, injection abstract, question set, and answer keys

2 weeks before

- [ ] Rehearse on 2–3 VMs: log in, run Lab A, sample in B1, build and connect a server, run the full Day 3 lab in hux
- [ ] Pre-run the Day 2 question set under all three methods and confirm the expected accuracy pattern
- [ ] Load-test paperKB and the workshop model with 25 concurrent clients

1 week before

- [ ] Provision all student VMs (up to 25); print credentials cards and the TA master sheet
- [ ] Refresh the Day 3 model list and lecture visuals
- [ ] Re-check the NIH notices; align the handout with IU's generative AI guidance

After the workshop

- [ ] Daily health checks from Day 1 through the week after Day 3; a shared channel for problems
- [ ] Send the shutdown reminder two days before; destroy the VMs; send the follow-up survey

Open decisions

Decision	Options	Default if undecided
Workshop dates and shutdown date	—	Needed for the allocation request and credentials cards
Workshop model	Jetstream-hosted model vs. a stronger REALLMS model for hux on Day 3	Jetstream model for all days, unless the Day 3 rehearsal shows unreliable tool calling
Logprobs for Day 1 B1	Live from the model vs. pre-captured	Pre-captured if the model endpoint does not return logprobs

Decision	Options	Default if undecided
Embedding model	Local small model vs. a Jetstream embedding endpoint	Local model with pre-cached weights
paperKB readiness	Live MCP server vs. dropped	If not ready, drop the paperKB-only question; everything else still runs
Day 3 main dataset	Bulk (GSE86221) vs. single-cell (MIX-seq)	Bulk, unless the survey shows a mostly single-cell cohort
hux on the VMs	Docker runtime; job tracking without a scheduler	Confirm in rehearsal

Sources

- Open WebUI: Model Context Protocol
- Jetstream2 education allocations
- NIH NOT-OD-25-132
- GSE86221 (OmicsDI)
- MIX-seq data via pertpy